



UNIVERSIDAD DE DAGUPAN
SCHOOL OF INFORMATION TECHNOLOGY EDUCATION
ITP10 | EVENT-DRIVEN PROGRAMMING
PRELIM | LABORATORY ACTIVITY 1

Name : Ed Andrew A. Rendon
Year, Course & blk : 31-ITE-02
Subject : ITP10 | EVENT-DRIVEN PROGRAMMING
Date : August 21, 2026

SCORE

Laboratory Activity #1: Basic Event-Driven Webpage

Objective:

At the end of the laboratory activity, students should be able to create a basic event-driven webpage using HTML, CSS, and JavaScript, apply `addEventListener()` to respond to user actions, identify event sources, events, listeners, and handlers, modify webpage content and appearance dynamically, and use the browser console to test and debug their program.

Activity Description

Create a simple interactive webpage that responds to different user actions. The webpage must demonstrate how JavaScript uses event listeners and event handlers to produce changes on a webpage.

Required Features

Your webpage must include the following:

- 1.
2. A heading with the exact `id="mainHeading"` that displays an initial message.
3. A text input with the exact `id="nameInput"` where the user can enter their name.
4. A Display Greeting button with the exact `id="greetingButton"` that changes the heading to a personalized greeting.
5. A Change Background button with the exact `id="backgroundButton"` that changes the webpage background color.
6. A Reset button with the exact `id="resetButton"` that restores the webpage to its original appearance.
7. A mouseover event that displays a message in the browser console.
8. An input event that shows the text being entered by the user.
9. At least one `console.log()` statement for debugging or event monitoring.
10. An external JavaScript file named `script.js`.
11. An external CSS file named `style.css`.

Suggested Interface

The webpage may contain:

- A title or heading
- A name input field
- A **Display Greeting** button
- A **Change Background** button
- A **Reset** button
- A paragraph where messages or entered text are displayed

Project Folder Structure

basic-event-page/

```
|— index.html
|— style.css
|— script.js
```

Instructions

Part A: Create the HTML Structure

Create an index.html file containing:

-
- A page container element with the exact id="pageContainer"
- A heading with the exact id="mainHeading"
- A text input with the exact id="nameInput"
- Three buttons with the exact ids: "greetingButton", "backgroundButton", and "resetButton"
- A paragraph or message area with the exact id="messageArea"
- Links to the external files using href="style.css" and src="script.js"

Important: Use the exact identifiers listed above. Do not rename them because the activity will be checked using automated tests.

Use the defer attribute when linking the JavaScript file.

```
<script src="script.js" defer></script>
```

Part B: Add the Design

In style.css, apply basic styling to make the webpage organized and readable.

Your CSS must include styling for:

-
- The page container
- Heading
- Input field
- Buttons
- Message area

Part C: Add Event-Driven Behavior

In script.js:

1. Select the required HTML elements using document.querySelector() with the exact selectors #pageContainer, #mainHeading, #nameInput, #greetingButton, #backgroundButton, #resetButton, and #messageArea.
2. Create separate handler functions for the greeting click, background click, reset click, input, and mouseover events.
3. Attach the handlers using addEventListener().
4. Do not use inline JavaScript such as onclick.
5. Add comments to identify the event source, event listener, and event handler.

Expected Behaviors

Display Greeting Button

When the button is clicked:

- Read the name entered by the user.
- Change the heading to:

Hello, [Name]!

- Display an appropriate message when the input is empty.

Change Background Button

When clicked:

- Change the webpage background color.
- Display a message indicating that the color was changed.

Reset Button

When clicked:

-
- Restore the original heading.
- Clear the input field.
- Restore the original background color.
- Clear the displayed message.

Input Event

While the user types:

- Display the current input below the text field.

Example:

You are typing: Maria

Mouseover Event

When the mouse pointer moves over one of the buttons:

- Display a message in the browser console.

Example:

The mouse is over the greeting button.

Coding Standards

-
- Use meaningful variable and function names, and keep all required HTML identifiers exactly as specified in Part A.
- Use const and let instead of var.
- Use proper indentation.
- Use semicolons consistently.
- Keep HTML, CSS, and JavaScript in separate files.
- Use `addEventListener()` for all events.
- Add brief and meaningful comments.
- Test the webpage using the browser console

Testing Checklist

Before submitting, verify that:

-
- All buttons work correctly.
- The greeting displays the entered name.
- Empty input is handled properly.
- The background color changes.
- The reset button restores the original page.
- The input event displays the current text.
- The mouseover event appears in the console.
- There are no errors in the browser console.
- The JavaScript and CSS files are correctly linked.

Submission Requirements

- Submit the complete project folder containing:
 - index.html
 - style.css
 - script.js
- Use the filename format:
Surname_FirstName_BasicEventPage.zip
- Include your name, course, section, and activity date on the webpage.

Grading Rubric

Criteria	Excellent	Good	Developing	Needs Improvement	Points
HTML Structure	Complete, correct, and well-organized HTML elements.	Mostly complete with minor errors.	Some required elements are missing.	Structure is incomplete or incorrect.	15
Design and Presentation	Clean, readable, and visually organized design.	Good design with minor issues.	Basic design with limited organization.	Poor or unfinished design.	10
Event Listeners and Handlers	All required events work correctly using addEventListener().	Most events work with minor errors.	Some events work, but several are incomplete.	Events are mostly missing or not working.	25
Required Functionality	All required features work correctly.	Most features work correctly.	Some features work with noticeable issues.	Most features are missing or incorrect.	25
Code Quality	Code is clean, properly indented, meaningful, and well-commented.	Code is readable with minor issues.	Code has several formatting or naming issues.	Code is difficult to read or poorly organized.	10
Testing and Debugging	No console errors and all test cases are completed.	Minor errors that do not affect major features.	Several errors affect some features.	Major errors prevent the webpage from working.	10
Submission Completeness	All files, screenshots, and required information are complete.	One minor requirement is missing.	Several requirements are missing.	Submission is incomplete.	5
Total					100

PREPARED BY:

ROCHELLE S. LANTO, MIT
SITE, FACULTY